



(/rol/app/)

Menu

Korean

홈 (/rol/app/) (home) > Observing OpenShift Service Mesh

> Chapter 1: Observing OpenShift Service Mesh

Observing OpenShift Service Mesh

Lesson

Lab Environment

Collecting Service Metrics

Objectives

- Identify the Prometheus metric types and their use cases for instrumenting applications.
- Explain how to instrument Node.js and Java Quarkus applications to expose custom metrics.
- Describe how OpenShift service monitor resources integrate application metrics with OpenShift user workload monitoring.
- Query and visualize service mesh and custom application metrics by using PromQL and the OpenShift web console.

Collecting Service Metrics

Observability is essential for operating and troubleshooting distributed applications in a service mesh. Metrics provide quantitative data about service health, performance, and behavior, enabling you to detect issues, optimize performance, and ensure reliable service delivery.

In this lecture, you learn how to instrument applications to expose custom metrics, configure OpenShift monitoring to collect those metrics, and query metrics data by using the OpenShift web console. You explore metrics instrumentation for both Node.js and Java applications, understand the role of service monitor resources, and practice writing Prometheus queries to analyze service behavior.

Metrics are numerical measurements that Prometheus collects over time to help you understand application and infrastructure performance, health, and behavior. In a service mesh environment, metrics provide visibility into request traffic, latency, error rates, and resource usage across all services in the mesh.

Red Hat OpenShift Service Mesh integrates with the OpenShift monitoring stack to provide comprehensive metrics collection and visualization capabilities.

Metrics in OpenShift

OpenShift provides two types of monitoring:

Platform monitoring

Collects metrics from cluster infrastructure components, such as nodes, control plane services, and cluster operators. OpenShift enables this monitoring by default, and cluster administrators manage it.

User workload monitoring

Collects metrics from user applications and namespaces. When enabled by a cluster administrator, this feature lets you monitor your own workloads and custom application metrics.

If an OpenShift administrator enables user workload monitoring, then you can query standard container and workload metrics, including the following examples:

- CPU usage: `container_cpu_usage_seconds_total`
- Memory usage: `container_memory_working_set_bytes`
- Container restarts: `kube_pod_container_status_restarts_total`
- Deployment replicas: `kube_deployment_status_replicas`

To learn more about the role-based access control configuration needed for regular users to view metrics, refer to the references section.

Service Mesh Metrics

When you add a namespace to OpenShift Service Mesh, the Envoy sidecar proxies automatically collect and emit metrics about network traffic flowing through the mesh. These metrics include:

- `istio_requests_total`: Total number of requests sent to a service.
- `istio_request_duration_seconds`: Time taken to process requests.
- `istio_request_bytes`: Size of HTTP request bodies.
- `istio_response_bytes`: Size of HTTP response bodies.

Service mesh metrics include labels that enable you to filter and aggregate data by different dimensions, such as:

- Source and destination workloads
- HTTP response codes
- Request protocols
- Service versions

These labels enable you to create queries that answer questions such as "What is the error rate for the `invoices` service?" or "What is the 95th percentile latency for requests to version 2 of the `stock` service?"

Prometheus Metric Types

Prometheus defines four core metric types. Understanding these types is essential for instrumenting your applications correctly.

Counter

A cumulative metric that only increases over time or resets to zero on restart. Use counters for values that always go up, such as the number of requests served, errors encountered, or tasks completed.

Gauge

A metric that can increase or decrease over time. Use gauges for values that fluctuate, such as memory usage, active connections, queue depth, or temperature readings.

Histogram

Records observations such as request durations or response sizes. A histogram counts those observations in configurable buckets. Histograms also provide a sum of all observed values. Use histograms when you need to calculate quantiles, also known as percentiles, or averages.

Summary

Similar to histograms, summaries record observations and provide percentiles. However, summaries compute quantiles on the client side, whereas histograms compute them at query time. Summaries are less commonly used than histograms in modern Prometheus deployments.



Figure 1.24: Prometheus metrics types

When creating custom metrics, choose the type that best matches your use case. For counting events, use a counter. For measuring current values that vary, use a gauge. For measuring distributions and calculating percentiles, use a histogram.

Instrumenting Node.js Applications with Prometheus Metrics

The implementation to emit metrics from your services depends on your application. Each programming language or development framework has its own way to implement telemetry. This lecture explains how to integrate Javascript Node.js and Java Quarkus

applications. Refer to the documentation of your programming language or framework to learn how to integrate them with OpenShift monitoring through Prometheus or OpenTelemetry.

To collect custom metrics from a Node.js application, use the `prom-client` library, which is the official Prometheus client for JavaScript.

First, install the library as a dependency in your project:

```
[user@host ~]$ npm install prom-client
```

Then, import and initialize the library in your application code. The following example demonstrates initialization with a custom prefix to namespace your metrics:

```
const prometheus = require('prom-client');
const prefix = 'invoices_svc';
prometheus.collectDefaultMetrics({ prefix });
```

The `collectDefaultMetrics()` function automatically collects standard Node.js runtime metrics, such as process CPU usage, memory usage, and event loop lag. The prefix parameter ensures that these metrics are namespaced to avoid conflicts with other applications.

Creating a Counter Metric

Counters track cumulative values that only increase. The following example creates a counter to track created invoices:

```
const invoices_num = new prometheus.Counter({
  name: 'invoices_svc:invoices_count',
  help: 'Number of created invoices'
});
```

To increment the counter when an event occurs:

```
invoices_num.inc(); // Increment by 1
invoices_num.inc(5); // Increment by 5
```

Creating a Gauge Metric for Response Time

Gauges measure values that can increase or decrease. The following example creates a gauge to track invoice creation response time:

```
const responseTime = new prometheus.Gauge({
  name: 'invoices_svc:invoices_creation_time',
  help: 'Time taken in seconds to create an invoice'
});
```

The `prom-client` library provides convenient methods to measure durations:

```
app.get('/create_invoice', async function (req, res) {
    responseTime.setToCurrentTime();
    const end = responseTime.startTimer();

    // Your application logic here
    await processRequest();

    end(); // Records the elapsed time
    res.send(response);
});
```

Exposing metrics for scraping

Prometheus collects metrics by periodically scraping an HTTP endpoint that returns metrics in the Prometheus text format. Create a `/metrics` endpoint in your application:

```
app.get('/metrics', async function (req, res) {
    res.set('Content-Type', prometheus.register.contentType);
    res.send(await prometheus.register.metrics());
});
```

This endpoint returns all registered metrics, including both custom metrics and default runtime metrics, in the format that Prometheus expects.

Instrumenting Quarkus Applications with Micrometer

Java applications running on Quarkus have different options for emitting metrics. This section focuses on Micrometer, which is the recommended approach for instrumenting Quarkus applications with custom metrics.

Quarkus supports three approaches for custom application metrics: SmallRye Metrics, Micrometer, and OpenTelemetry. Understanding the differences helps you choose the right approach for your project.

The following table compares these three approaches. The Quarkus SmallRye Metrics implementation is deprecated, but still in use in many enterprise deployments.

Feature	SmallRye Metrics	Micrometer	OpenTelemetry
Extension	quarkus-smallrye-metrics	quarkus-micrometer + quarkus-micrometer-registry-prometheus	quarkus-opentelemetry
Metrics endpoint	/metrics or /q/metrics	/q/metrics	Configurable (OTLP or Prometheus exporter)
Instrumentation style	Annotation-based	Annotation-based	Programmatic API
Status	Deprecated in Quarkus 3.x	Current recommended approach	Future-focused, unified observability

Feature	SmallRye Metrics	Micrometer	OpenTelemetry
Backend support	Prometheus only	Multiple backends (Prometheus, InfluxDB, Datadog...)	Multiple backends via OTLP (Prometheus, Jaeger, vendors)
Traces integration	No	No (separate tracing libraries needed)	Yes (unified metrics, traces, and logs)
Learning curve	Low (simple annotations)	Low (simple annotations)	Medium (programmatic API)
Flexibility	Low	Medium	High (full control over metric properties)

In this training content, you focus on the Micrometer metrics library. Micrometer is a vendor-neutral metrics facade that provides a simple API for instrumenting code and supports multiple monitoring systems, including Prometheus.

To enable Micrometer in a Quarkus application, add the following dependencies to your `pom.xml`:

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-micrometer</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-micrometer-registry-prometheus</artifactId>
</dependency>
```

The `quarkus-micrometer` extension provides the core metrics functionality, and `quarkus-micrometer-registry-prometheus` adds Prometheus format support and automatically creates a `/q/metrics` endpoint.

Micrometer provides annotations that you can apply to methods to automatically collect metrics without writing manual instrumentation code.

Using the `@Counted` Annotation

The `@Counted` annotation from Micrometer increments a counter each time the annotated method is invoked. This is useful for tracking how many times an operation occurs:

```
import io.micrometer.core.annotation.Counted;

@GET
@Path("/invoice")
@Counted(value = "invoices_requested",
        description = "count of invoices requested")
public String createInvoice() {
    // Create invoice logic
    return "Invoice created";
}
```

Each time a client calls the `/invoice` endpoint, the `invoices_requested` counter increments by one. Micrometer automatically exports this metric with the `_total` suffix, so in Prometheus it appears as `invoices_requested_total`.

You can also use `@Counted` to track events within a method, not just method invocations. For example, you might count how many times a specific condition occurs:

```
@Counted(value = "invoice_process_rating",
        description = "Overall customer rating for the invoice process")
Integer getCalculatedRating() {
    return calculateRating();
}
```

Each time you call this method, it increments the `invoice_process_rating_total` counter.

You can control counters programmatically without using annotations. For example, you can use a counter for invoices created successfully, and another one for failed invoices, as the following code shows:

```

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import jakarta.annotation.PostConstruct;
import jakarta.inject.Inject;

@Inject
MeterRegistry registry;

private Counter invoicesCreated;
private Counter invoicesFailed;

@PostConstruct
void initMetrics() {
    invoicesCreated = Counter.builder("invoices_created_total")
        .description("Total number of invoices successfully created")
        .register(registry);

    invoicesFailed = Counter.builder("invoices_failed_total")
        .description("Total number of failed invoice creation attempts")
        .register(registry);
}

@GET
@Path("/invoice")
public String createInvoice() {
    try {
        // Business logic to create invoice
        invoicesCreated.increment();
        return "Invoice created";
    } catch (Exception e) {
        invoicesFailed.increment();
        return "Invoice failed";
    }
}

```

Using the @Timed Annotation

The @Timed annotation measures the duration of method executions and creates histogram data that you can use to calculate percentiles and averages:

```

import io.micrometer.core.annotation.Timed;

@GET
@Path("/invoice")
@Timed(value = "invoice_process_time",
    description = "A measure of how long it takes to process an invoice")
public String createInvoice() {
    // Create invoice logic
    return "Invoice created";
}

```

When you apply @Timed to a method, Micrometer records:

- The total number of invocations (invoice_process_time_seconds_count)
- The total time spent in the method (invoice_process_time_seconds_sum)

- A histogram with duration buckets (`invoice_process_time_seconds_bucket`)

These metrics enable you to calculate:

- Average execution time: `sum / count`
- Request rate: `rate(count[5m])`
- 95th percentile latency: `histogram_quantile(0.95, ...)`

Unlike a simple gauge that shows the current value, `@Timed` gives you statistical distribution data over time.

Combining Multiple Annotations

You can stack multiple metric annotations on a single method to collect different aspects of its behavior:

```
@GET
@Path("/invoice")
@Counted(value = "invoice_creation_requested",
         description = "count of invoice creation requested")
@Timed(value = "invoice_process_time",
       description = "A measure of how long it takes to process an invoice")
public String createInvoice() {
    // Create invoice logic
    return "Invoice created";
}
```

This approach creates two separate metrics from a single endpoint:

- `invoice_creation_requested_total` - A counter tracking total invocations
- `invoice_process_time_seconds_*` - Histogram data tracking execution duration

This combination gives you comprehensive observability into both how often callers invoke the endpoint and how long it takes to respond.

Accessing Metrics in Quarkus

Micrometer automatically creates a `/q/metrics` endpoint that exposes all metrics in Prometheus text format. The endpoint includes:

- JVM metrics: Memory pools, garbage collection, thread counts, and class loading
- System metrics: CPU usage, file descriptors, and uptime
- Application metrics: Your custom metrics defined with annotations
- HTTP server metrics: Request counts, durations, and status codes

The metrics endpoint returns metrics in Prometheus format, including help text, type declarations, and the actual metric values.

Alternative Quarkus Instrumentation with OpenTelemetry

Quarkus also supports OpenTelemetry, an open-source observability framework that provides a vendor-neutral API for collecting metrics, traces, and logs. Unlike Micrometer's annotation-based approach, OpenTelemetry^{4/9} uses a programmatic API for instrumenting

applications. OpenTelemetry is particularly useful when you need unified observability across metrics, traces, and logs, or when your organization is standardizing on OpenTelemetry.

For detailed information about instrumenting Quarkus applications with OpenTelemetry, refer to the references section.

Configuring Metrics Collection Resources

OpenShift provides two types of monitoring resources for collecting metrics: `ServiceMonitor` and `PodMonitor`. Understanding when to use each resource is essential for comprehensive service mesh observability.

ServiceMonitor

Monitors pods through Kubernetes Service resources. Use `ServiceMonitor` to collect custom application metrics from your application's metrics endpoint exposed through a Kubernetes Service.

PodMonitor

Monitors pods directly without requiring a Kubernetes Service. Use `PodMonitor` to collect metrics from Envoy sidecar proxies in service mesh environments. The Envoy proxy exposes metrics at the `/stats/prometheus` endpoint on each pod.

In OpenShift Service Mesh environments, you typically use both resources:

- `PodMonitor` for collecting service mesh metrics from Envoy sidecar proxies
- `ServiceMonitor` for collecting custom application metrics from your application code

The following example shows a `PodMonitor` resource that collects metrics from Envoy sidecar proxies:

```

apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: istio-proxies-monitor
spec:
  selector:
    matchExpressions:
      - key: istio-prometheus-ignore
        operator: DoesNotExist
  podMetricsEndpoints:
    - path: /stats/prometheus
      interval: 30s
      relabelings:
        - action: keep
          sourceLabels: [__meta_kubernetes_pod_container_name]
            regex: "istio-proxy"
            action: keep
          sourceLabels: [__sourceLabels: [__meta_kubernetes_pod_annotationpresent_prometheus_io_scrape]
            - action: replace
              regex: (\\d+);((( [A-Fa-f0-9]{1,4}::?){1,7}[A-Fa-f0-9]{1,4})
              replacement: '[$2]:$1'
              sourceLabels: [__meta_kubernetes_pod_annotation_prometheus_io_port, meta_kube
rnetes_pod_ip]
              targetLabel: address
            - action: replace
              regex: (\\d+);((( [0-9]+?)(\\.|$)){4})
              replacement: $2:$1
              sourceLabels: [__meta_kubernetes_pod_annotation_prometheus_io_port, meta_kube
rnetes_pod_ip]
              targetLabel: address
            # Set the 'app' label from 'app.kubernetes.io/name' or fallback to 'app'
            - sourceLabels: ["__meta_kubernetes_pod_label_app_kubernetes_io_name", "meta_ku
bernetes_pod_label_app"]
              separator: ";"
              targetLabel: "app"
              action: replace
              regex: "(.+);.|.;(.)"
              replacement: "${1}${2}" # Use the first non-empty value
            # Set the 'version' label from 'app.kubernetes.io/version' or fallback to 've
rsion'
            - sourceLabels: ["__meta_kubernetes_pod_label_app_kubernetes_io_version", "meta
_kubernetes_pod_label_version"]
              separator: ";"
              targetLabel: "version"
              action: replace
              regex: "(.+);.|.;(.)"
              replacement: "${1}${2}" # Use the first non-empty value
            # add some labels we want
            - sourceLabels: [__meta_kubernetes_namespace]
              action: replace
              targetLabel: namespace
            - action: replace
              replacement: "ossm-3"
              targetLabel: mesh_id

```

This pod monitor resource enables Prometheus to scrape Istio service mesh metrics from the Envoy sidecar proxies, providing visibility into request traffic, latency, and error rates across all services in the mesh.

IMPORTANT

The pod monitor resource must be on each namespace that belongs to the service mesh.

This lecture focuses on ServiceMonitor for collecting custom application metrics.

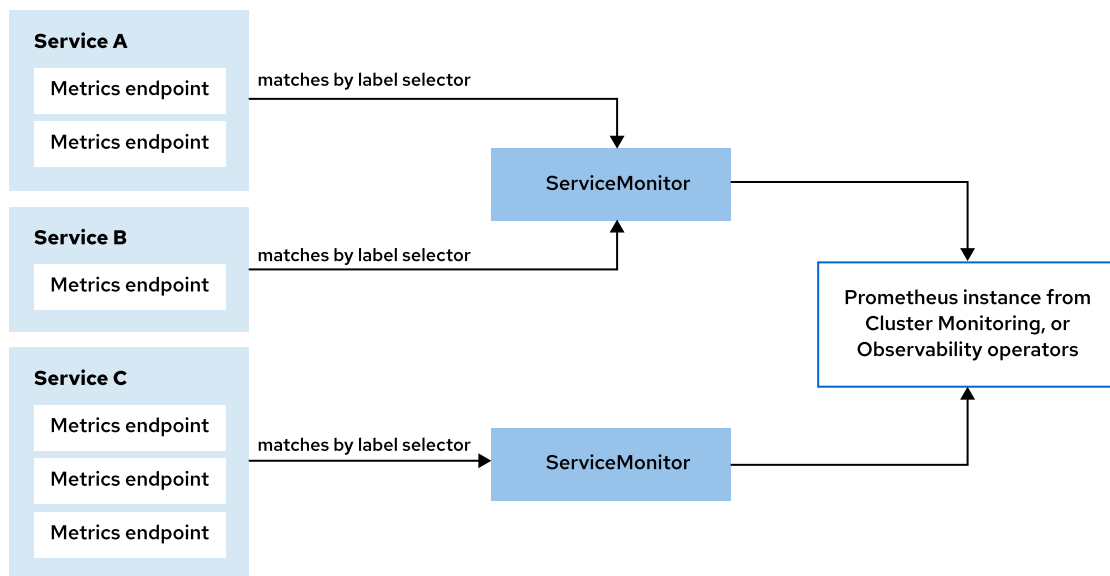


Figure 1.25: Relationship between Services, ServiceMonitors, and Prometheus

A ServiceMonitor references a Kubernetes Service and defines scraping parameters:

```
apiVersion: monitoring.coreos.com/v1 ❶
kind: ServiceMonitor
metadata:
  name: invoices-monitor
  namespace: my-project
spec:
  endpoints:
    - interval: 30s ❷
      path: /metrics ❸
      scheme: http ❹
      targetPort: 8080 ❺
  selector:
    matchLabels:
      app: invoices ❻
```

- ❶ Use `monitoring.observability.openshift.io/v1alpha1` if using a customized monitoring stack with the Observability Operator.

- ❷ Scrape every 30 seconds.

- 3 HTTP path to scrape.
- 4 Use HTTP, not HTTPS.
- 5 Port on the pods.
- 6 Match services with this label.

For Quarkus applications using Micrometer, the path must point to `/q/metrics`:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: stock-monitor
  namespace: my-project
spec:
  endpoints:
    - interval: 30s
      path: /q/metrics
      scheme: http
      targetPort: 8080
  selector:
    matchLabels:
      app: stock
```

1 Quarkus Micrometer endpoint

The `selector.matchLabels` field must match the labels on your Kubernetes Service resource. The monitoring operator uses this selector to identify which services to monitor.

For applications running in the service mesh with sidecar injection, ensure that:

- The `targetPort` matches the application container port, not the Envoy sidecar port
- The path points to your application's metrics endpoint
- The namespace has user workload monitoring enabled

After you create the `ServiceMonitor`, the OpenShift monitoring stack automatically begins scraping metrics from matching services. You can then query these metrics in the OpenShift web console.

Using the OpenShift Web Console for Inspecting Metrics

The OpenShift web console provides built-in tools for exploring metrics without needing external tools.

To access metrics:

1. Log in to the OpenShift web console
2. Go to the Developer perspective
3. Click **Observe** → **Metrics**
4. Select your project from the project dropdown

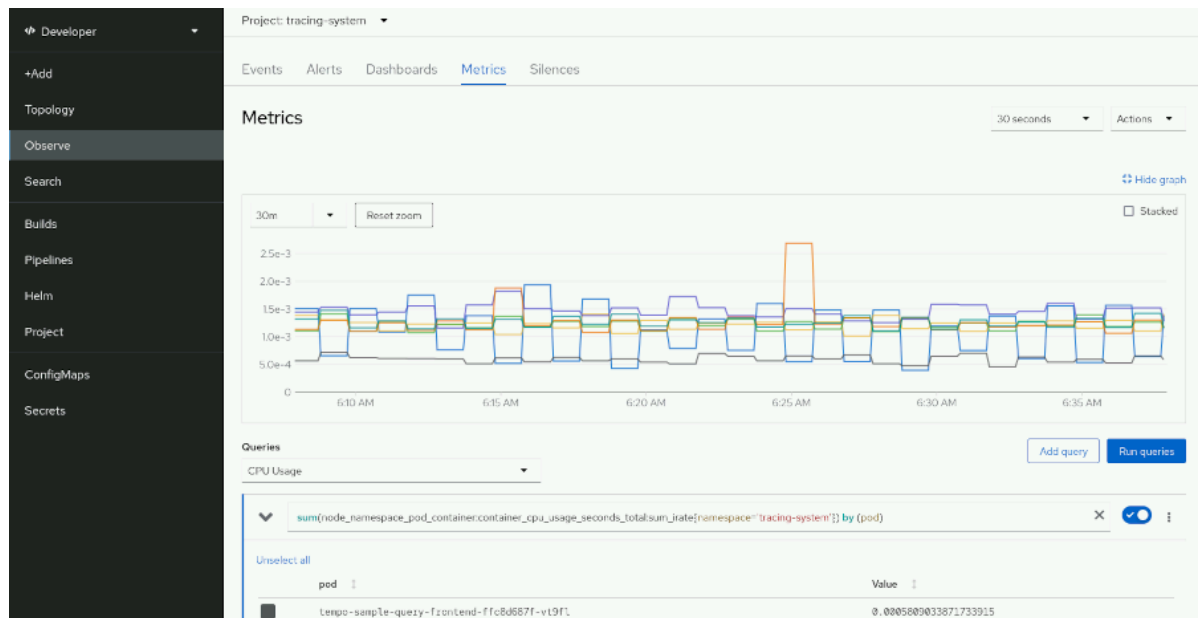


Figure 1.26: The metrics tab in the Observe menu from the OpenShift console developer perspective

The Metrics page provides:

Query interface

Enter PromQL queries in the expression field and click **Run queries** to execute them. There are some common pre-defined queries.

Time range controls

Select the time window for your query.

Graph and table views

Visualize metrics as time-series graphs or view raw data in table format.

Multiple query series

Add multiple queries to the same graph to compare different metrics or services.

Resolution controls

Adjust the sampling interval for your query results.

You can compare metrics across multiple services by adding multiple queries to the diagram. To do that, perform the following steps:

1. Enter the first query and click **Run queries**.
2. Click **Add query** to create a second query field.
3. Enter the second query and click **Run queries** again.
4. Both metrics appear on the same graph for comparison.

The console automatically color-codes different query series and provides a legend that shows which line corresponds to each metric.

For custom application metrics to appear in the console, remember to ensure the following prerequisites:

- The cluster administrator enables user workload monitoring on the cluster.
- Your application exposes metrics at a `/metrics` or `/q/metrics` endpoint.
- You create and correctly configure a `ServiceMonitor` resource.
- The `ServiceMonitor` selector matches your Kubernetes `Service` labels.
- The `ServiceMonitor` path matches your application's metrics endpoint.
- Your application pods are running and healthy.

If you fulfill these prerequisites, then metrics become available within one scrape interval, usually 30 seconds.

Querying Metrics with Promql

Prometheus Query Language (PromQL) is a language that enables you to select, filter, and aggregate time-series metrics data. The OpenShift web console provides a query interface at **Observe** → **Metrics**.

The following are examples of common PromQL queries. You can use these queries in the **Expression** field in the metrics view in the OpenShift console.

Retrieve a metric by name

```
invoices_svc:invoices_count
```

This query returns the current value of the counter for all instances of the `invoices` service.

Filter by label values

```
istio_requests_total{destination_workload="stock"}
```

This query returns request counts only for the `stock` workload.

Because counters always increase, graphing them directly is not useful. Instead, use the `rate()` function to calculate the per-second increase over a time window.

Calculate the rate of change for counters

```
rate(istio_requests_total{reporter="source"}[5m])
```

This query shows the request rate per second over the last five minutes. The `[5m]` range vector selects data points from the last five minutes.

Calculate rate for application metrics

```
rate(invoices_created_total[5m])
```

This query computes the rate of invoices created per second. Application metrics from Micrometer have the `_total` suffix.

Aggregate metrics across multiple instances

```
sum(rate(istio_requests_total{reporter="source"}[5m])) by (destination_workload)
```

This query aggregates metrics by using the `sum()` and `by()` functions. It performs the following operations:

1. Calculates the rate of requests per second
2. Sums the rates across all pod instances
3. Groups results by `destination_workload`

The result shows total requests per second for each workload.

Calculate percentiles from histogram metrics

```
histogram_quantile(0.95,  
  sum by (le, destination_workload) (  
    rate(istio_request_duration_milliseconds_bucket{reporter="source"}[5m])  
  )  
)
```

This query uses the `histogram_quantile()` function to compute percentiles from histogram metrics. It calculates the 95th percentile request latency in milliseconds for each destination workload. The result shows that 95% of requests complete faster than this duration.

Calculate average duration from a timed metric

```
rate(invoice_process_time_seconds_sum[5m]) / rate(invoice_process_time_seconds_co  
unt[5m])
```

To get the mean duration, this query calculates the average duration by dividing the total time spent by the number of invocations.

Calculate error rate percentage

```
100 * sum(rate(istio_requests_total{reporter="source", response_code=~"5.."}[5m]))  
by (destination_workload)  
/  
sum(rate(istio_requests_total{reporter="source"}[5m])) by (destination_workload)
```

This query calculates the error rate percentage. It performs the following operations:

1. Calculates the rate of 5xx error responses, which are server errors
2. Divides by the total request rate
3. Multiplies by 100 to convert to a percentage
4. Groups results by destination workload

The `response_code=~"5.."` uses a regular expression to match all HTTP status codes in the 500–599 range.

To learn more about PromQL, refer to the references section.

REFERENCES

For more information about OpenShift Monitoring, refer to the *Monitoring* documentation at

https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index

(https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index)

For more information about role-based access control to monitoring user projects, refer to the

Granting users permissions for monitoring for user-defined projects section in the *Configuring user workload monitoring* chapter of the *Monitoring* guide at

https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index#granting-users-permission-to-monitor-user-defined-projects_preparing-to-configure-the-monitoring-stack-uwm

(https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index#granting-users-permission-to-monitor-user-defined-projects_preparing-to-configure-the-monitoring-stack-uwm)

For more information about OpenShift Service Mesh metrics configuration, refer to the *Metrics and Service Mesh* chapter in the *Observability* guide in the OpenShift Service Mesh documentation at

https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/observability/index#metrics-and-service-mesh

([https://docs.redhat.com/en/documentation/red_hat_openshift_service_mes h/3.1/html-single/observability/index#metrics-and-service-mesh](https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/observability/index#metrics-and-service-mesh))

prom-client - Prometheus client for Node.js (<https://github.com/siimon/prom-client>)

Micrometer Metrics - Quarkus (<https://quarkus.io/guides/micrometer>)

OpenTelemetry - Quarkus (<https://quarkus.io/guides/opentelemetry>)

How Quarkus works with OpenTelemetry on OpenShift

(<https://developers.redhat.com/articles/2025/07/07/how-quarkus-works-opentelemetry-openshift>)

Knowledge Base: How to use Prometheus Query Language (PromQL) in OpenShift (<https://access.redhat.com/articles/7067755>)



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트
(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책
(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침
(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정